



Security Review For Meteora



Collaborative Audit Prepared For:
Lead Security Expert(s):

Meteora
bin2chen
carrotsmuggler

Date Audited:

November 27 - December 9, 2025

Introduction

Meteora's Dynamic Vaults are DeFi's first dynamic yield infrastructure, where the vaults rebalance every minute across lending platforms to find the best possible yield while prioritizing keeping user funds as accessible as possible.

Scope

Repository: [sherlock-scoping/MeteoraAg__mercurial-vault](https://github.com/sherlock-scoping/MeteoraAg__mercurial-vault)

Audited Commit: [6bbfbfd9527bfabef8e449f278b53bb2d11e274b](https://github.com/sherlock-scoping/MeteoraAg__mercurial-vault/commit/6bbfbfd9527bfabef8e449f278b53bb2d11e274b)

Final Commit: [0ec3f3fb847832ce175666ad7efd38eb7b16e9f3](https://github.com/sherlock-scoping/MeteoraAg__mercurial-vault/commit/0ec3f3fb847832ce175666ad7efd38eb7b16e9f3)

Files:

- `libs/jup_lend/lending_reward_rate_model/src/constants.rs`
- `libs/jup_lend/lending_reward_rate_model/src/extensions/lending_reward_rate_model.rs`
- `libs/jup_lend/lending_reward_rate_model/src/extensions/mod.rs`
- `libs/jup_lend/lending_reward_rate_model/src/lib.rs`
- `libs/jup_lend/lending/src/constants.rs`
- `libs/jup_lend/lending/src/helpers.rs`
- `libs/jup_lend/lending/src/lib.rs`
- `libs/jup_lend/library/src/lib.rs`
- `libs/jup_lend/library/src/math/bn.rs`
- `libs/jup_lend/library/src/math/casting.rs`
- `libs/jup_lend/library/src/math/ceil_div.rs`
- `libs/jup_lend/library/src/math/floor_div.rs`
- `libs/jup_lend/library/src/math/mod.rs`
- `libs/jup_lend/library/src/math/safe_math.rs`
- `libs/jup_lend/library/src/math/tick.rs`
- `libs/jup_lend/library/src/math/u256.rs`
- `libs/jup_lend/library/src/structs.rs`
- `libs/jup_lend/library/src/token/mod.rs`
- `libs/jup_lend/library/src/token/spl.rs`
- `libs/jup_lend/liquidity/src/constants.rs`
- `libs/jup_lend/liquidity/src/extensions/mod.rs`

- `libs/jup_lend/liquidity/src/extensions/rate_model.rs`
- `libs/jup_lend/liquidity/src/extensions/token_reserve.rs`
- `libs/jup_lend/liquidity/src/lib.rs`
- `libs/jup_lend/liquidity/src/seeds.rs`
- `programs/vault/src/instructions/add_strategy.rs`
- `programs/vault/src/instructions/claim_rewards.rs`
- `programs/vault/src/instructions/deposit_withdraw_liquidity.rs`
- `programs/vault/src/instructions/enable_vault.rs`
- `programs/vault/src/instructions/get_unlocked_amount.rs`
- `programs/vault/src/instructions/initialize_idle_vault.rs`
- `programs/vault/src/instructions/initialize.rs`
- `programs/vault/src/instructions/initialize_strategy.rs`
- `programs/vault/src/instructions/mod.rs`
- `programs/vault/src/instructions/rebalance_strategy.rs`
- `programs/vault/src/instructions/remove_strategy2.rs`
- `programs/vault/src/instructions/remove_strategy.rs`
- `programs/vault/src/instructions/set_operator.rs`
- `programs/vault/src/instructions/transfer_admin.rs`
- `programs/vault/src/instructions/transfer_fee_vault.rs`
- `programs/vault/src/instructions/update_locked_profit_degradation.rs`
- `programs/vault/src/lib.rs`
- `programs/vault/src/macros.rs`
- `programs/vault/src/seed.rs`
- `programs/vault/src/state.rs`
- `programs/vault/src/strategy/base.rs`
- `programs/vault/src/strategy/jup.rs`
- `programs/vault/src/strategy/mod.rs`
- `programs/vault/src/strategy/vault.rs`
- `programs/vault/src/utils.rs`

Final Commit Hash

0ec3f3fb847832ce175666ad7efd38eb7b16e9f3

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	3	5

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: rebalance strategy lacks slippage protection [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/53>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

When executing `withdraw_strategy()`, the current calculation of the required collateral amount involves the following `min()` operation:

```
pub fn handler_withdraw_strategy<'a, 'b, 'c: 'info, 'info>(
...
    let max_collateral_amount = strategy_handler.get_collateral_amount(
        &ctx.accounts.collateral_vault,
        reserve,
        remaining_accounts,
        strategy.key(),
    )?;
    rebalance_strategy_wrapper(
        move || {
            let collateral_amount = strategy_handler.from_liquidity_to_collateral(
                reserve,
                remaining_accounts,
                amount,
                Rounding::Up,
            )?;
@>    let collateral_amount = collateral_amount.min(max_collateral_amount);

    if collateral_amount != 0 {
        strategy_handler.withdraw(
            strategy_program,
            vault_account,
            token_vault,
            collateral_vault,
            reserve,
            token_program,
            strategy_account,
            remaining_accounts,
            &token_mint,
            vault_seeds,
            &strategy_bumps,
            collateral_amount,
        )?;
```

Executing `min()` may result in the actual retrieved amount being significantly lower than expected. (Potential slippage protection failure)

For example: `strategy.exchange_rate = 1 : 1` `strategy.collateral_vault.shares_amount = 100`

1. admin intends to withdraw all liquidity, offline calculation: `amount = 100`
 - thus calls `withdraw_strategy(amount = 100)`
2. During transaction execution, due to security incidents or other factors, `exchange_rate` plummets to: `exchange_rate= 100 : 1`
 - so:
 - `collateral_amount = from_liquidity_to_collateral(100) = 100_00`
 - `collateral_amount = min(1000_00, 100) = 100`
 - `liquidity = 1`

Therefore: The transaction ultimately succeeds. Although `amount = 100` was specified, the final amount received is 1.

Impact

The core reason lies in the lack of slippage protection mechanisms within `withdraw_strategy/deposit_strategy`. Currently, the online calculation of exchange rates cannot effectively mitigate slippage.

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/rebalance_strategy.rs#L148

Recommendation

1. Add slippage parameters to `deposit_strategy()/withdraw_strategy()`: `min_deposit_shares/max_burn_shares`

Or

2. Using `libs::jup_lend` to compute `exchange_rate` in real-time fails to adequately detect slippage anomalies because:
 - If it's a security incident, then the live computation is inherently flawed
 - `jup do` Minor adjustments to calculation `exchange_rate` logic could render `withdraw_from_lending_with_reserve_checks()/deposit_to_lending_with_reserve_checks()` ineffective

Therefore, recommend adding a configuration: `Strategy.except_exchange_rate` then check the real-time exchange rate within a 1% fluctuation range. admin periodically synchronize reasonable `except_exchange_rate` values from jup.

Discussion

codewithgun

Hi, we will acknowledge this

Issue M-2: profit unlocking may be maliciously delayed [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/54>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Detail

Each rebalance will execute `update_locked_profit()`

```
pub fn rebalance_strategy_wrapper<'c: 'info, 'info>(<
...
    vault
        .locked_profit_tracker
@>        .update_locked_profit(gain, loss, current_time)
        .ok_or(VaultError::MathOverflow)?;
...
    }

pub fn update_locked_profit(&mut self, gain: u64, loss: u64, current_time: u64)
-> Option<()> {
    let mut locked_profit = self.calculate_locked_profit(current_time)?;
    if loss > 0 {
        if loss < locked_profit {
            locked_profit -= loss;
        } else {
            locked_profit = 0;
        }
    }
    if gain > 0 {
        locked_profit = locked_profit.checked_add(gain)?;
    }
@>    self.last_updated_locked_profit = locked_profit;
@>    self.last_report = current_time;
    Some(())
}
```

in `update_locked_profit()`, it will update: `last_updated_locked_profit = locked_profit`
`last_report = current_time`

this may potentially causing profit unlocking to be continuously delayed.

Using `withdraw_directly_from_strategy()` as an example:

Assume: `strategy_1` = Strong profitability `strategy_2` = Weak profitability `last_report` = 12:00 `last_updated_locked_profit` = 100 (Most of the profits come from `strategy_1`)

locked_profit_degradation = 6 hours

1. current_time = 15:00

- so unlocked_profit = 50 , locked_profit = 50 (fully unlocked after 3 hours)

2. use maliciously call `withdraw_directly_from_strategy(strategy_2)`, but `strategy_2` the profit is very small `gain = 1`

- so last_report = 15:00, last_updated_locked_profit = 51 (fully unlocked after 6 hours)

Meaning: the original old profit 50 unlock in 3 hours now requires 6 hours to unlock.

Impact

Profit unlocking may be maliciously delayed

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/remove_strategy.rs#L230

Recommendation

Do not recalculate gain/loss when executing rebalance. Add a separate function for periodically refreshing profits: `report_profit()`.

Or

gain mix in last_updated_locked_profit:

```
new_last_updated_locked_profit = old_last_updated_locked_profit + gain
new_last_report = (old_last_report * old_last_updated_locked_profit + current_time * gain) / new_last_updated_locked_profit
```

```
from keep unlock: (current_time - new_last_report) * new_last_updated_locked_profit = (current_time - old_last_report) * old_last_updated_locked_profit
```

Discussion

codewithgun

Hi, we will acknowledge this because it's by design.

Issue M-3: share ratio can be inflated to block participation [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/58>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Funds can be deposited into the vault without minting any shares. This inflates the ratio of assets to shares in the system and can be used to prevent users from participating.

Vulnerability Detail

During deposits, the `lp_token_amount` is calculated from based on the current assets and `lp supply`, rounded down.

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/deposit_withdraw_liquidity.rs#L133-L135

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/state.rs#L160-L164

However, this mint value can also be 0, since there are no checks to prevent that. Furthermore, the vault can have any amount of assets and `lp shares` minted, since there is no locking of `LP shares`. This allows an attacker to inflate the share ratio when the vault is empty, to block future participants.

The attacker first mints 1 `LP share` by depositing 1 `lamport` of a token.

The attacker then needs to have some interest accrue, so that the contract holds 2 `lamports` of assets with 1 share. This can be done by depositing a large amount, having it lent out into `jupiter` to accrue interest, and then removing all but one share of the funds. Due to the rounding down, the system will have 1 share and 2 assets (or more).

In such a state, the attacker can then deposit 1 asset, and this will mint 0 shares, while bumping up the total assets to 3. Next, they can deposit 2 assets to mint 0 shares, bumping up the total assets to 5. Next, they can deposit 4 assets to mint 0 shares, bumping up the total assets to 9.

The total asset progressively grows by 2^n for each deposit, all backed by one share. Thus, the single share can be made to represent thousands of tokens.

Impact

This can actually block other participants, or force them to eat large rounding errors. If 1 lp share is backed by thousands of assets, other participants also need to deposit thousands of assets in order to get any lp shares, or they risk getting 0 and losing their deposit. Thus, users cannot participate with lower amounts of funds. Also the rounding errors will be in the order of 1 share, which will be worth thousands of tokens.

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/state.rs#L160-L164

Tool Used

Manual Review

Recommendation

Consider burning or locking the initial 1000 shares minted by the system. This will prevent the inflation attack since it depends on rounding errors, whose impact will be minimized.

Discussion

codewithgun

Hi, we'll acknowledge this issue because we'll not be adding any new vaults with lending from now on. All the existing vaults will be in maintenance mode (no new feature / update to it)

Issue L-1: when initialize strategy, missing check for other_bumps [RESOLVED]

Source:

<https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/51>

Vulnerability Detail

When initializing the strategy, we set `strategy.bumps`.

```
pub fn handler_initialize_strategy<'a, 'b, 'c: 'info, 'info>(
    ctx: Context<'a, 'b, 'c, 'info, InitializeStrategy<'info>>,
    bumps: StrategyBumps,
    strategy_type: StrategyType,
) -> Result<()> {
    msg!("set strategy {:?}", strategy_type);
    let strategy = &mut ctx.accounts.strategy;
    strategy.reserve = ctx.accounts.reserve.key();
    strategy.collateral_vault = ctx.accounts.collateral_vault.key();
    strategy.strategy_type = strategy_type;
    @> strategy.bumps = bumps.other_bumps;
```

Currently, no checks are performed on `bumps.other_bumps`. This value is used in `strategy_handler.deposit/withdraw` to generate `strategy_seeds`.

```
    let strategy_bumps = strategy.bumps;

    fn withdraw<'c: 'info, 'info>(
        ...
        let strategy_index = strategy_bumps[0];
        let strategy_bump = strategy_bumps[1];

        let strategy_seeds = [
            vault.key.as_ref(),
            reserve.key.as_ref(),
            &[strategy_index],
            &[strategy_bump],
        ];
```

It is recommended to add a check, avoid being unable to execute during deposit/withdraw

Impact

unable to execute during deposit/withdraw

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/initialize_strategy.rs#L66

Recommendation

```
pub fn handler_initialize_strategy<'a, 'b, 'c: 'info, 'info>(  
    ...  
    strategy.strategy_type = strategy_type;  
    strategy.bumps = bumps.other_bumps;  
+   assert!(bumps.other_bumps[0] == bumps.strategy_index);  
+   assert!(bumps.other_bumps[1] == ctx.bumps.strategy);
```

Discussion

codewithgun

Hi, it's fixed in <https://github.com/MeteoraAg/mercurial-vault/pull/309>

bin2chen66

Fixed in [PR-309](#)

add check bumps.other_bumps[0]/[1]

Issue L-2: when burn user LP, redundant signer_seeds are provided [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/52>

Vulnerability Detail

When executing `withdraw()`, `token::burn()` is called to burn the user's LP tokens.

```
pub fn handler_withdraw(
...
    let vault_seeds = vault_seeds!(vault);
    let signer = &[&vault_seeds[..]];
...
    token::burn(
        CpiContext::new_with_signer(
            ctx.accounts.token_program.to_account_info().clone(),
            Burn {
                mint: ctx.accounts.lp_mint.to_account_info().clone(),
                from: ctx.accounts.user_lp.to_account_info().clone(),
                authority: ctx.accounts.user.to_account_info().clone(),
            },
            @> signer,
        ),
        unmint_amount,
    )?;
    Ok(())
}
```

This method still passes `signer_seeds == vault_seeds` as an argument, which is not utilized. It is recommended not to pass this argument to avoid potential security risks that may arise from subsequent code modifications.

Note: `handler_withdraw_directly_from_strategy()` similar

Impact

Subsequent code modifications may introduce other security risks.

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/deposit_withdraw_liquidity.rs#L224

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/deposit_withdraw_liquidity.rs#L383

Recommendation

```
token::burn(  
    CpiContext::new_with_signer(  
        ctx.accounts.token_program.to_account_info().clone(),  
        Burn {  
            mint: ctx.accounts.lp_mint.to_account_info().clone(),  
            from: ctx.accounts.user_lp.to_account_info().clone(),  
            authority: ctx.accounts.user.to_account_info().clone(),  
        },  
        signer,  
        &[],
```

Discussion

codewithgun

Hi, it's fixed in <https://github.com/MeteoraAg/mercurial-vault/pull/308>

bin2chen66

Fixed in [PR-308](#)

`withdraw()/withdraw_directly_from_strategy()` both have removed the unnecessary `signer seed`.

Issue L-3: token_vault_advance_payment must not be strategy.collateral_vault [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/55>

Vulnerability Detail

The current restrictions for token_vault_advance_payment are as follows:

```
/// token_vault_advance_payment
/// the account must be different from token_vault
/// the owner of token_advance_payment must be vault
#[account(mut, constraint = token_vault_advance_payment.key() != token_vault.key())]
pub token_vault_advance_payment: Account<'info, TokenAccount>,
```

To prevent token_vault from being misused, but another extreme scenario is still missing: Using token_vault_advance_payment = strategy.collateral_vault. (if vault.token_mint == strategy.collateral_vault.mint)

Example: vault = {token_mint = token_a} strategy_1 = { collateral_vault_a } =>
collateral_vault_a.mint = fToken strategy_2 = { collateral_vault_b } =>
collateral_vault_b.mint = token_a

then call remove_strategy2(strategy = strategy_1, token_vault_advance_payment = strategy_2.collateral_vault_b)

This allows malicious exploitation of strategy_2.collateral_vault_b

Impact

Malicious exploitation of other strategies' collateral_vault

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/remove_strategy2.rs#L46

Recommendation

Given this highly unlikely scenario vault.token_mint == strategy.collateral_vault.mint and the method requires privileged access, it is recommended to add a code comments (as a reminder of possibilities) instead of making code changes.

Discussion

codewithgun

Hi, it's fixed in <https://github.com/MeteoraAg/mercurial-vault/pull/307>

bin2chen66

Fixed in [PR-307](#)

Added comment

Issue L-4: update_locked_profit_degradation() may delay the unlocking of generated profits [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/56>

This issue has been acknowledged by the team but won't be fixed at this time.

Vulnerability Details

When executing update_locked_profit_degradation(), we call update_locked_profit():

```
pub fn handler_update_locked_profit_degradation(
...
    vault
        .locked_profit_tracker
        .update_locked_profit(0, 0, current_time)
...
}

pub fn update_locked_profit(&mut self, gain: u64, loss: u64, current_time: u64)
    -> Option<> {
    ↪ let mut locked_profit = self.calculate_locked_profit(current_time)?;
...
    if gain > 0 {
        locked_profit = locked_profit.checked_add(gain)?;
    }
    @> self.last_updated_locked_profit = locked_profit;
    @> self.last_report = current_time;
    Some(())
}
```

Updating the new locked_profit_degradation may cause the unlocking speed of old profits to be significantly slower than users expect.

For example: Current: last_report = 12:00, last_updated_locked_profit = 100, locked_profit_degradation = 6 hours

1. current_time = 15:00
 - unlocked_profit = 50 , locked_profit = 50 (fully unlocked after 3 hours)
2. admin call update_locked_profit_degradation(locked_profit_degradation = 12 hours)
 - last_report = 15:00, last_updated_locked_profit = 50 (fully unlock delayed to 12 hours later)

Meaning: The original profit 50 unlock in 3 hours now requires 12 hours to unlock.

Impact

The unlocking of generated profits has been postponed.

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/instructions/update_locked_profit_degradation.rs#L34

Recommendation

add delayed activation mechanism, without affecting the unlocking of already generated profits.

Discussion

codewithgun

Hi, we will acknowledge this issue because it's by design.

Issue L-5: `get_collateral_exchange_rate()` may be maliciously specified with incorrect `supply_token_reserve_liquidity` [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/issues/57>

Vulnerability Detail

Currently, `get_collateral_exchange_rate()` performs the following checks to prevent users from maliciously specifying incorrect accounts:

```
fn get_collateral_exchange_rate<'info>(  
    lending: &'info AccountInfo<'info>,  
    f_token_mint: &'info AccountInfo<'info>,  
    supply_token_reserve_liquidity: &'info AccountInfo<'info>,  
    reward_rate_model: &'info AccountInfo<'info>,  
) -> Result<u64> {  
    let token_reserve_account =  
        AccountLoader::<TokenReserve>::try_from(supply_token_reserve_liquidity)?;  
  
    let token_reserve_state = token_reserve_account.load()?;  
    let lending_state = Account::<Lending>::try_from(lending)?;  
    require!(  
        lending_state.f_token_mint.eq(&f_token_mint.key())  
        @>    && lending_state.mint.eq(&token_reserve_state.mint)  
        && lending_state  
            .rewards_rate_model  
            .eq(&reward_rate_model.key()),  
        VaultError::InvalidAccountsForStrategy  
    );  
}
```

When verifying `supply_token_reserve_liquidity`, only check that `supply_token_reserve_liquidity.mint == lending_state.mint`.

If multiple `TokenReserve` instances exist with identical mint values, malicious actors could specify an invalid `supply_token_reserve_liquidity`, resulting in incorrect exchange rates.

`supply_token_reserve_liquidity` is rechecked during `jupstrategy_handler.deposit/withdraw`, but `remove_strategy2()` does not invoke this check, creating a security vulnerability.

Impact

Executing `remove_strategy2()` may be exploited to specify a malicious `supply_token_reserve_liquidity`, thereby compromising exchange rate calculations.

Code Snippet

https://github.com/sherlock-audit/2025-11-meteora-dynamic-vaults-nov-27th/blob/a4fbee8fcf94eb39be9909e308237eb540052cc5/MeteoraAg__mercurial-vault/programs/vault/src/strategy/jup.rs#L161

Recommendation

```
require!(
    lending_state.f_token_mint.eq(&f_token_mint.key())
+    &&
↪ lending_state.token_reserves_liquidity.eq(&supply_token_reserve_liquidity.key)
    && lending_state.mint.eq(&token_reserve_state.mint)
    && lending_state
        .rewards_rate_model
        .eq(&reward_rate_model.key()),
    VaultError::InvalidAccountsForStrategy
);
```

Discussion

codewithgun

Hi, it's fixed in <https://github.com/MeteoraAg/mercurial-vault/pull/306>

bin2chen66

Fixed in [PR-306](#)

```
add check lending_state.token_reserves_liquidity == supply_token_reserve_liquidity
```

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.